resiDNA_workflows

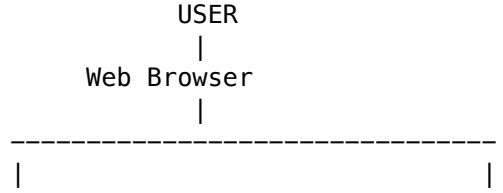
Residual DNA qPCR Analysis — Colab Notebook and Local Streamlit App

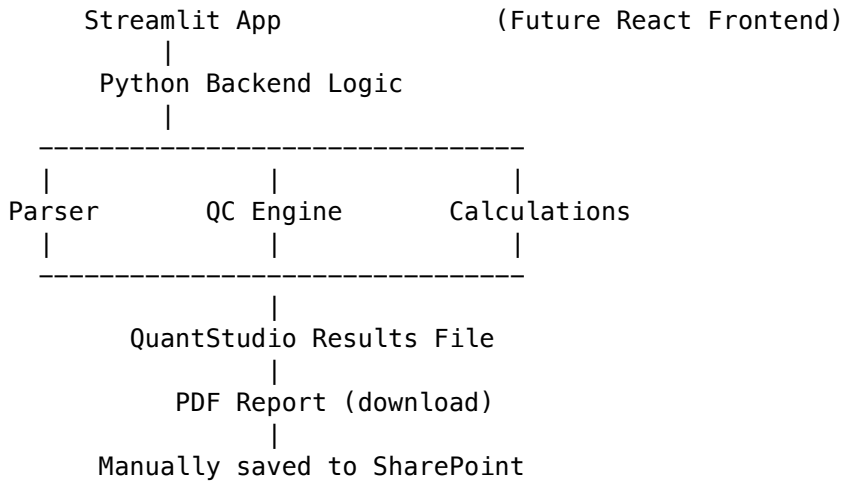
Project Organization

```
resiDNA_workflows/
│
├── app/
│   ├── app.py                    # Phase II Streamlit app — local web UI, mirrors
↪  the notebook
│   ├── audit_log.py               # opt-in audit log (RESIDNA_AUDIT_LOG) —
↪  app.py-only, see Data Privacy below
│   └── requirements.txt           # dependencies for the Streamlit app
├── .streamlit/
│   └── config.toml                # light theme + wide layout (must stay at repo
↪  root)
│
├── Notebooks/
│   └── 01_Phase I.ipynb          # Phase I notebook — import through PDF report
↪  generation
│
├── Data/
│   ├── experiment_001.xlsx       # primary dataset the notebook was built against
│   ├── sample_data_001.xls       # real-world dataset — validated
│   └── sample_data_002.xls       # real-world dataset — validated
│
├── Scripts/
│   ├── qpcr.py                    # shared classification/suitability logic —
↪  imported by both the notebook and app.py
│   └── plotting.py                # shared style_table() — imported by both the
↪  notebook and app.py
├── Figures/                      # reserved — future shared modules
├── Results/                      # generated PDF reports
├── Logs/                        # audit.log, only if RESIDNA_AUDIT_LOG is enabled
↪  — gitignored, not committed
├── README.md
├── IT_Deployment_Guide.md        # IT handoff: on-prem / Azure + SharePoint
↪  hosting paths (Option A decided)
└── IT_Setup_Guide_OptionA.md     # from-scratch runbook IT executes: commands,
↪  service files, config templates
```

Application Architecture (In Progress)

Design of a local-first architecture, transitioning to IT-hosted per IT_Deployment_Guide.md.





Development Road Map

Phase I - Colab Scientific Prototype (.ipynb) >Prove that the qPCR analysis logic works correctly. 1. [x] Load QuantStudio file 2. [x] QuantStudio Results Parser 3. [x] Data Classification Engine 4. [x] System Suitability Engine 5. [x] Sample Result Processing 6. [x] Create Prototype Graphs 7. [x] Validate against `Data/sample_data_001.xls` and `Data/sample_data_002.xls` 8. [x] PDF Report generation

Phase II - Convert Prototype into a Local Application (http://localhost:8501) >Create a simple website you run on your own computer using Local Streamlit App. 1. [x] Upload Page 2. [x] Backend Processing 3. [x] Results Dashboard

Phase III - Add Criteria Management >Stop changing code every time acceptance criteria change. >Update thresholds directly in the notebook's `#@param` cell and mirror them into `app.py` — no separate Settings UI or database needed.

Phase IV - Improve Reporting and Visualization — Not needed >Current interactive Plotly charts and the PDF report already meet the bar for a professional analytical tool; dropped from the roadmap.

Phase V - Hand Off to Company IT — Next >Move from prototype to maintainable, IT-hosted software. >Pin dependencies, organize into a professional VS Code project, and transition hosting to company-controlled infrastructure per IT_Deployment_Guide.md (on-prem/local server or Azure + SharePoint embed) with Entra ID SSO.

Phase VI - (Maybe) Upgrade to Full Web Application >Only worth revisiting if IT's hosted deployment outgrows what Streamlit can offer (e.g. multi-user concurrency, custom auth flows beyond OIDC). >Replace Streamlit with React Frontend and FastAPI Backend

Acceptance Criteria

STD (Standard Curve) | Check | Formula | Pass | |—|—|—| | R^2 | instrument-computed | ≥ 0.99 | | Ct %CV (triplicate) | `(Ct_SD / Ct_Mean) × 100` | ≤ 20% | | Back-calculation bias | `\|(Back-Calc Qty − Nominal Qty) / Nominal Qty\| × 100` | ≤ 25% | | Amplification efficiency | `(10^(−1/Slope) − 1) × 100` | 90%–110% |

ERC & PC (Controls) | Check | Formula | Pass | |—|—|—| | Quantity %CV (triplicate) | `(Qty_SD / Qty_Mean) × 100` | ≤ 20% | | PC %Recovery | `(Dilution-Adjusted Qty / Reference-Adjusted Qty) × 100` | 80%–125% | | ERC %Recovery | same formula | 50%–150% |

Sample LOQ / STD Range — replaces the old ICH-sigma LOQ formula. From the standard curve's own back-calculated values, take STD1 (highest nominal quantity) and STD6 (lowest): | Value | Source | Used for | |—|—|—| | LOQ (Quantity) | STD6 `Back-Calc Mean` | Sample STD Range / "below LOQ" floor | | LOQ (Ct) | STD6 `Ct Mean` | NTC / NEC pass check | | STD Range | `[STD6 Back-Calc Mean, STD1 Back-Calc Mean]` | Sample STD Range check |

NTC / NEC — both pass per well if Ct is `Undetermined` or `Ct ≥ LOQ_Ct` (STD6's own measured Ct Mean above).

Sample Suitability — evaluated in this order, since each step feeds the next:

1. Triplicate single-outlier exclusion: if a dilution's full 3-well Quantity %CV fails 25%, `resolve_sample_replicates()` (`Scripts/qpcr.py`) drops the outlier well and re-checks the remaining 2; if they pass, that pair's mean becomes the dilution's Quantity %CV / Quantity Mean / Total DNA / DNA per Protein ("Replicates Used: 2") instead of an outright fail. Only triggers on an already-failing triplicate — passing ones keep the instrument's reported figures as-is. Every check below uses this resolved `Quantity Mean`.
2. Quantity %CV: `(Qty_SD / Qty_Mean) × 100` ≤ 25%.
3. STD Range: the resolved `Quantity Mean` (pre-dilution-adjustment) must fall within the STD Range above; below it is `Out of Range` and is "below LOQ", above it is also `Out of Range`.
4. Dilutional Linearity: `%Bias` $= (x_1 - x_2) / x_2 \times 100 \leq 20\%$, where x_2 is the `Dilution Adjusted` quantity of the sample's own dilution with the lowest combined Ct %CV + Quantity %CV among dilutions that already pass both step 2 and step 3, and x_1 is each dilution's `Dilution Adjusted` quantity. A sample with no dilution passing steps 2–3 has no valid reference, so every one of its dilutions gets `N/A`.
5. Combined Suitability: `Pass` only if steps 2–4 all pass; otherwise `Fail` (binary — no `N/A`, since an uninterpretable dilution isn't a soft middle ground). Only `Pass` dilutions get averaged into the final result.

Two tables: Final Sample Results by Dilution and Final Sample Results — Averaged per Sample (every sample is still reported, even one with zero passing dilutions — averaged across its passing dilutions if any exist, otherwise across all of them as a flagged fallback). In the averaged table: red highlight = the sample failed acceptance criteria (takes priority), green highlight = passed and `DNA per Protein` is below `SAMPLE_DNA_PER_PROTEIN_LIMIT` (15 ng/mg), gray + `- LOQ` suffix on `Total DNA` / `DNA per Protein` = the averaged `Quantity Mean` is below the LOQ (low confidence, footnoted). `Sample #` (e.g. `S1`) groups dilutions into the averaged table but isn't itself displayed in the by-dilution table — `Sample` (e.g. `S1 D1`) already carries that information.

Next Step — `compute_sample_status()` (`Scripts/qpcr.py`) turns the by-dilution Suitability breakdown into a next-action label per sample: | Next Step | Meaning | |—|—| | `Reportable` | At least one dilution cleanly passes all of steps 2–4 above (same condition as `Sample Passed`). | | `LOQ - Spike Test` | Quantity %CV and Dilutional Linearity pass (Linearity `N/A` counts as non-blocking), but STD Range fails because the dilution's `Quantity Mean` is below STD6 — below the assay's LOQ. | | `ULOQ - Adjust Dilution` | Same as above, but STD Range fails because `Quantity Mean` is above STD1 — above the curve's highest calibrated point. | | `Retest` | Everything else: STD Range passes but Quantity %CV or Linearity fails, or multiple checks fail together. |

Each dilution is classified individually first, then a sample with multiple dilutions reports whichever status ranks best across them (`Reportable` > `LOQ - Spike Test` / `ULOQ - Adjust Dilution` > `Retest`) — the same "any dilution passing is enough" rule already used for `Sample Passed`.

Criteria Verification

- Output Verification: all thresholds live in one Colab-form-editable cell (`#@param`) in the notebook, right after Data Classification. Values marked "instrument-computed" above are read forward from the QuantStudio export as-is (not recalculated) — the notebook's Output Verification cell checks they

survive parsing unchanged. The one exception is Dilutional Linearity's `% Bias`, which is genuinely computed rather than read from the instrument.
- Manual Golden-Reference Check: a one-time sanity check for that computed `% Bias` value. Hand-calculate (or Excel-calculate) the expected `% Bias` for a few known dilutions against the reference-dilution formula, enter them into the notebook's `GOLDEN_LINEARITY_BIAS` dict, and re-run the cell — it flags any dilution where the notebook's computed value doesn't match your expected value within a `±0.01` percentage-point tolerance (`GOLDEN_BIAS_TOLERANCE`).

PDF Report

Both the notebook and the Streamlit app generate the same formatted PDF summarizing a run — Run Info, System Suitability, Sample Suitability, Final Sample Results, and a Signatures block. In the notebook, this is the final `## PDF Report` section, using the `#@param` fields in the Report Info cell; it writes to `Results/Sample_Analysis_Report_<run_no>.pdf` and requires `reportlab` (auto-installs if missing). In the app, it's built the same way but in-memory and offered as a browser download instead.

- ☐ The logo is currently a placeholder box in both — swap in a real logo file.

Streamlit App (DEMO)

Demo deployment: https://residna.streamlit.app — hosted on Streamlit Community Cloud, no installation needed, demo use only. Access is restricted to authorized company viewers (see Data Privacy & Security before uploading real company data). See below to run it locally instead.

`app.py` is the Phase II local web app: upload a QuantStudio file and get the same System Suitability, Sample Suitability, and Final Sample Results sections as the notebook, without touching code, plus: - Sample ID / Sample Name inputs — one row per Base Sample actually detected in the upload (simpler than the notebook's 8 fixed `#@param` slots, since the app can generate inputs from real data). - Standard Curve graphs (QC-controls overlay and per-sample overlay) and Amplification Curves (ΔRn vs Cycle) — all interactive Plotly charts. - A Download PDF Report button producing the same report described above.

Only the sections that map to the PDF report are shown — raw file preview, parse stats, and the classification table run silently in the background rather than cluttering the page.

Data Privacy & Security (DEMO Streamlit App)

In transit - The Community Cloud link is served over HTTPS by default — traffic between the browser and Streamlit's servers is encrypted.

In the app - `app.py` processes uploaded files entirely in memory (`st.file_uploader` → pandas parsing → Plotly charts / PDF generation). By default, nothing is written to disk or a database on the server — no `Results/` writes, no SQLite. Once a session ends or the app restarts, the uploaded file and its analysis are gone. - The PDF report is streamed straight to the browser as a download; it isn't retained server-side. - Exception — audit log: `app/audit_log.py` appends one record per generated PDF report (timestamp, uploaded filename, a SHA-256 hash of it, assay/run info, submitter/reviewer name, pass/fail — never the analytical results themselves) to `Logs/audit.log`, but only if the `RESIDNA_AUDIT_LOG` environment variable is set. It's unset (off) by default and on this demo deployment, so the "nothing persisted" behavior above still holds here — it's only enabled on the company-hosted deployment, where compliance requires it (see IT_Deployment_Guide.md).

Who can reach the app - Access to the live link is restricted via Streamlit Community Cloud's viewer authentication: in the app's dashboard under Settings → Sharing, it is set to an allowlist of company email addresses — only people who sign in with one of those emails can open it. - Residual risk worth knowing: even restricted, uploaded data still passes through Streamlit's shared Community Cloud infrastructure, which is intended for demos/public apps rather than as a substitute for company-controlled hosting. For the most sensitive runs, run the app locally instead (see below) so data never leaves your machine or network.

For moving to a fully company-hosted deployment (on-prem/local server or Azure + SharePoint embed), see IT_Deployment_Guide.md.

Running the Notebook Locally

The notebook auto-detects Colab vs. local execution (`IN_COLAB` flag) — outside Colab it skips `git clone`/`pip install` `klib`/`google.colab` imports and resolves paths relative to the repo instead of `/content/`.

```
python -m venv .venv
.venv\Scripts\pip install pandas numpy openpyxl xlrd klib ipykernel plotly
 ↪  reportlab
.venv\Scripts\python -m ipykernel install --user --name resiDNA-venv
 ↪  --display-name "resiDNA (.venv)"
```

Then select the resiDNA (.venv) kernel in VS Code / Jupyter and run all cells.

Running the Streamlit App Locally

Run these from the repo root — `.streamlit/config.toml` only applies when Streamlit is launched from the directory that contains it.

```
python -m venv .venv
.venv\Scripts\pip install -r app\requirements.txt
.venv\Scripts\streamlit run app\app.py
```

Opens at `http://localhost:8501` — only reachable while that command is still running in a terminal; closing it (or restarting your computer) stops the app until you run it again.

The audit log (see Data Privacy above) is off by default locally too. To try it: set `RESIDNA_AUDIT_LOG=1` before launching (`$env:RESIDNA_AUDIT_LOG="1"` in PowerShell), generate a report, then check `Logs/audit.log`.

Running Tests

```
python -m venv .venv
.venv\Scripts\pip install -r app\requirements.txt -r tests\requirements.txt
.venv\Scripts\pytest tests -v
```

Three suites, all under `tests/`: - `test_qpcr.py` — unit tests for every pure function in `Scripts/qpcr.py`, independent of any data file. - `test_app_golden.py` — runs the real `app/app.py` (headlessly, via a fake `streamlit` module in `tests/streamlit_stub.py`) against each file in `Data/` and checks `final_results/reportable_results` against known-good values, so a future change that silently shifts a Sample Suitability result gets caught automatically. `Data/*.xls/.xlsx` are real (scrubbed) company data and aren't committed to the repo (see Data Privacy above and git history), so these tests skip, not fail, wherever those files aren't present — a fresh clone or CI run will show them skipped, which is expected. - `test_audit_log.py` — unit tests for `app/audit_log.py`, including that it's a no-op unless `RESIDNA_AUDIT_LOG` is set and that raw analytical data never ends up in a log record.

Backlog — Recommended Cleanup

- ☐ Notebook outputs (styled tables, the `klib` plot) are committed inline, which makes diffs large. Consider `nbstripout` or committing a rendered HTML export alongside a stripped notebook.
- ☐ `Scripts/qpcr.py` / `Scripts/plotting.py` now hold the shared, self-contained logic (`style_table`, `classify_sample`, `recovery_bounds`, `combined_suitability`, the LOD sigma-Ct formula, the single-outlier triplicate resolution (`resolve_sample_replicates`), the Dilutional Linearity %Bias

calculation, the Sample LOQ/STD Range derivation, STD Range suitability, the averaged-results fallback aggregation, and the per-sample Next Step classification) that both the notebook and `app.py` import. Deliberately left duplicated: the parsing/cleaning steps and suitability-table assembly, since the notebook keeps those inline, cell-by-cell, on purpose — that step-by-step visibility is Phase I's whole point (see Road Map). Worth revisiting only if that tradeoff stops being worth it.